

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний університет «Львівська політехніка»
Інститут комп'ютерних наук та інформаційних технологій
Кафедра «Системи штучного інтелекту»



ЛАБОРАТОРНА РОБОТА №1

з дисципліни

«Штучний інтелект та інноваційні інформаційні технології»

на тему

Побудова та аналіз моделей штучного інтелекту з використанням TensorFlow

Виконав:

студент групи ШІ-42

Галич Максим

Викладач:

доц. каф. СШІ к.т.н.,

Думин І.Б.

Львів — 2026

Мета роботи. Опанувати практику створення й оптимізації моделей машинного навчання у TensorFlow, навчитися застосовувати різні архітектури, регуляризацію та методи покращення узагальнюючої здатності моделі, виконати порівняльний аналіз результатів.

1. Вибір набору даних

Для роботи обрано набір даних **California Housing** (перепис населення США 1990 року), доступний через `sklearn.datasets.fetch_california_housing`. Набір містить **20 640** записів і **8** числових ознак; цільова змінна `MedHouseVal` — медіанна вартість будинку в блоці, виміряна в сотнях тисяч доларів. Таким чином, розв’язується **задача регресії**.

Табл. 1: Ознаки набору даних California Housing

Ознака	Зміст
MedInc	Медіанний дохід домогосподарств у блоці
HouseAge	Медіанний вік будинків
AveRooms	Середня кількість кімнат на домогосподарство
AveBedrms	Середня кількість спалень на домогосподарство
Population	Населення блоку
AveOccup	Середня кількість мешканців на домогосподарство
Latitude	Географічна широта
Longitude	Географічна довгота

2. Розвідувальний аналіз даних

2.1. Розподіл ознак

Гістограми розподілу всіх ознак та цільової змінної наведено на рисунку 1. Ознаки `AveRooms`, `AveBedrms`, `Population` та `AveOccup` мають виражені довгі «хвости» з поодинокими екстремальними значеннями, тоді як `Latitude` і

Longitude демонструють двомодальний розподіл, що відповідає двом великим агломераціям штату — Лос-Анджелесу та району затоки Сан-Франциско.

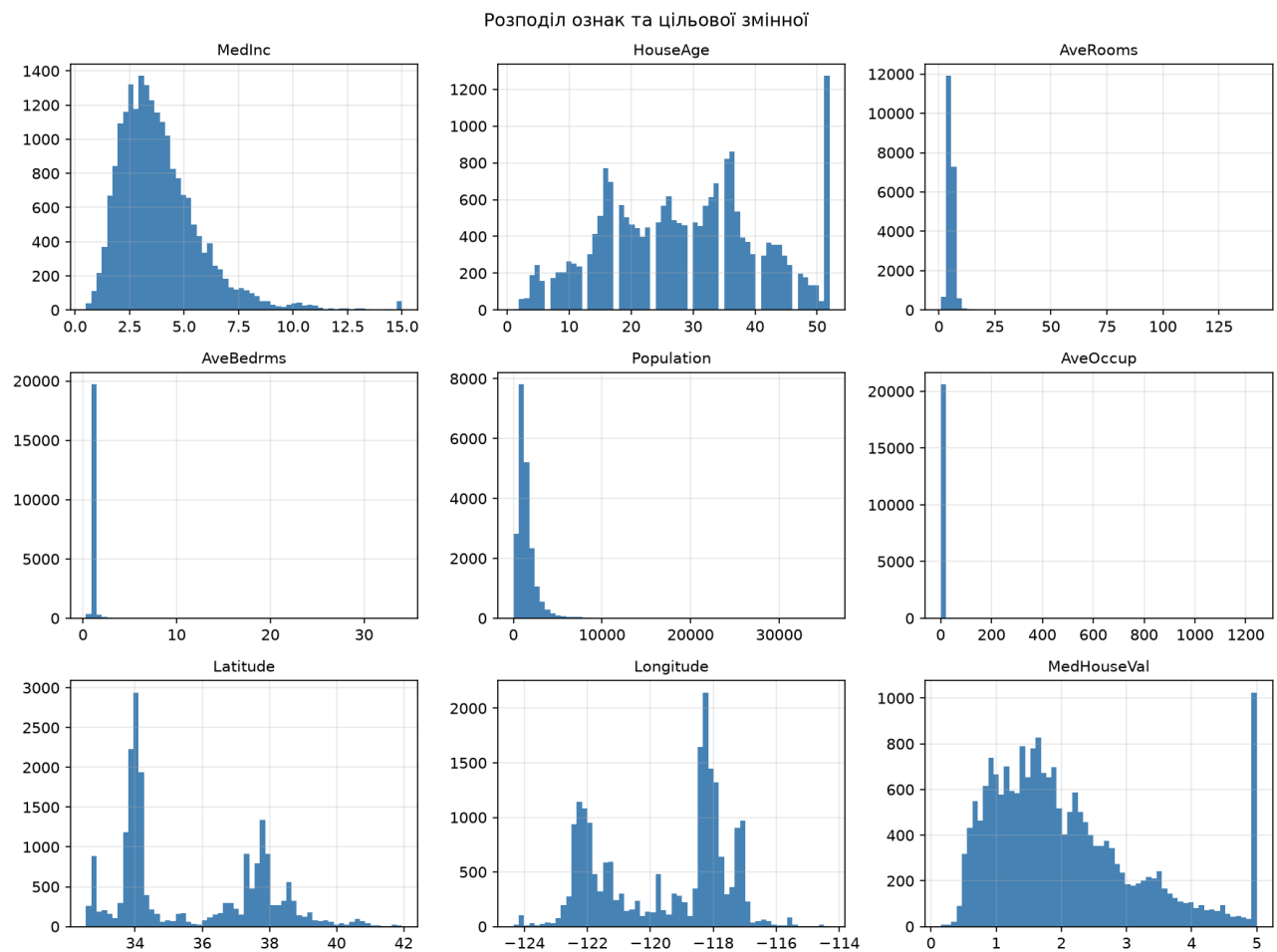


Рис. 1: Розподіл ознак та цільової змінної

2.2. Кореляційний аналіз

Кореляційну матрицю Пірсона зображено на рисунку 2. Найсильніший зв'язок із цільовою змінною має медіанний дохід: $r(\text{MedInc}, \text{MedHouseVal}) = 0,689$. Решта ознак корелює з ціною значно слабше (за модулем не більше 0,152). Окремо варто відзначити дуже високу кореляцію між AveRooms та AveBedrms ($r = 0,848$), що вказує на мультиколінеарність і додатково обґрунтовує застосування методу головних компонент.

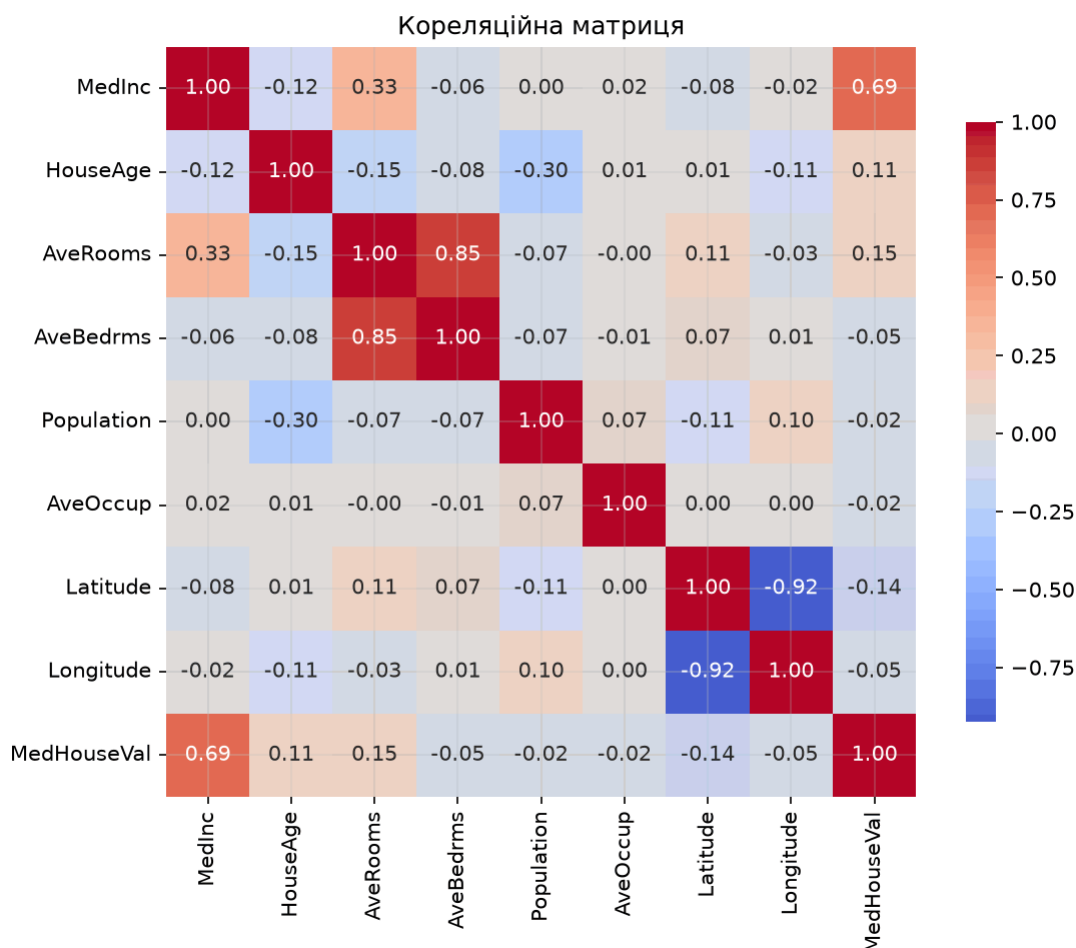


Рис. 2: Кореляційна матриця ознак та цільової змінної

2.3. Аналіз дисбалансу

Оскільки розв'язується задача регресії, класового дисбалансу в прямому розумінні не існує. Натомість проаналізовано два споріднені ефекти, які так само здатні зміщувати навчання (рисунок 3).

По-перше, розподіл цільової змінної є **правосторонньо асиметричним**: коефіцієнт асиметрії складає 0,978. По-друге, виявлено **штучне обмеження зверху**: рівно 992 записи (4,81 % вибірки) мають значення цільової змінної 5,00001, тобто їхню справжню вартість було «обрізано» на межі 500 тис. доларів ще під час формування набору даних. Ці записи принципово не можуть бути передбачені точно жодною моделлю і формують нижню межу досяжної похибки.

Для коректного врахування перекосу цільову змінну розбито на 5 квантильних груп, за якими виконано **стратифікований** поділ вибірки. Завдяки цьому частка дорогих та дешевих об'єктів у тренувальній, валідаційній і тестовій вибірках збігається.

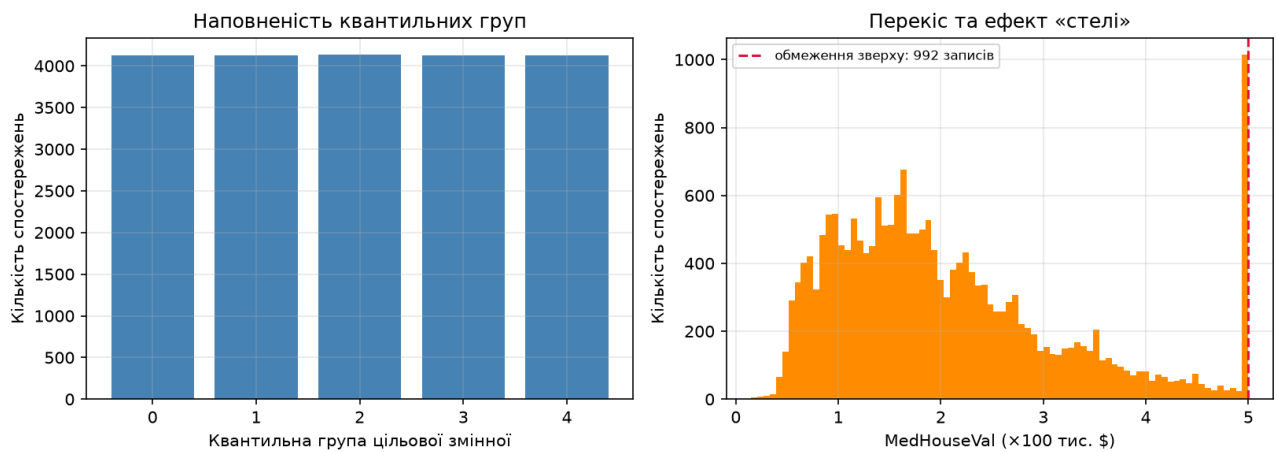


Рис. 3: *Перекіс розподілу цільової змінної та ефект обмеження зверху*

3. Попередня обробка даних

3.1. Стандартизація

Ознаки мають принципово різні масштаби (Population сягає десятків тисяч, тоді як AveBedrms коливається близько одиниці), тому застосовано StandardScaler. Критично важливо, що параметри масштабування оцінюються **виключно на тренувальній вибірці** і лише потім застосовуються до валідаційної та тестової — інакше статистики тестових даних просочилися б у процес навчання (data leakage).

3.2. Метод головних компонент

Результат PCA наведено на рисунку 4. Для збереження 90 % дисперсії потрібно **5 з 8** компонент. Перша компонента пояснює лише 25,3 % дисперсії, тобто дані не мають однієї домінуючої осі мінливості, а інформація розподілена між ознаками досить рівномірно. З огляду на це для навчання моделей використано **повний набір з 8 ознак**: відкидання трьох компонент коштувало б 10 % дисперсії, не даючи помітного виграшу у швидкості на вибірці такого розміру.

Додатково виконано нелінійну проєкцію t-SNE на випадковій підвибірці з 3000 записів. Проєкція показує, що об'єкти з близькою вартістю формують зв'язні області, тобто ознаковий простір дійсно несе інформацію, достатню для регресії.

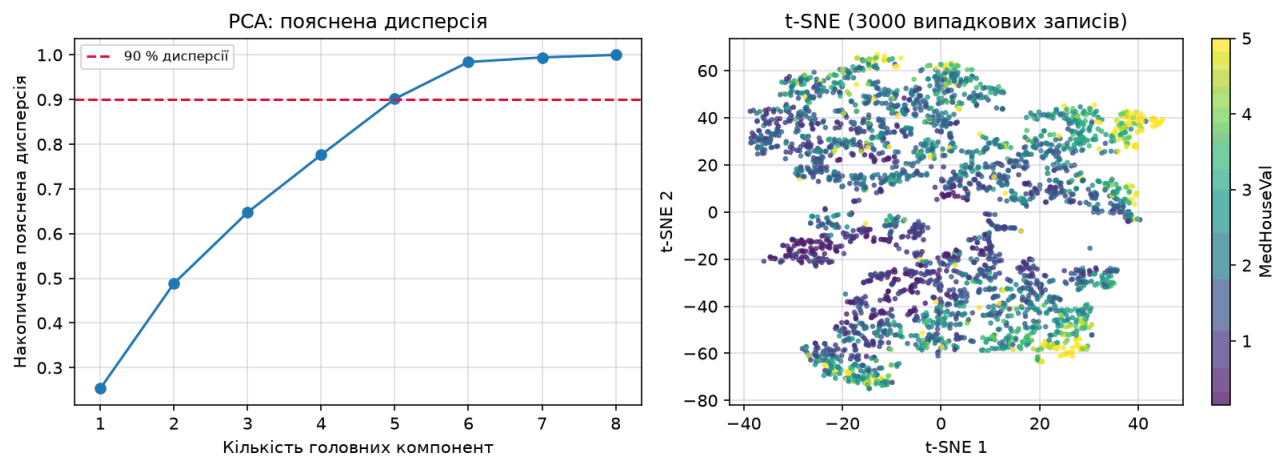


Рис. 4: *PCA: накопичена пояснена дисперсія та t-SNE-проекція*

3.3. Поділ вибірки

Вибірку поділено у співвідношенні **60/20/20** зі стратифікацією за квантильними групами цільової змінної: 12 384 записи для навчання, 4 128 для валідації та 4 128 для тестування.

```

32 def split(X: pd.DataFrame, y: pd.Series):
33     """A 60/20/20 split stratified by the quantile groups of the target.
34
35     Stratification is needed because the distribution of MedHouseVal is
36     noticeably skewed and artificially capped at its maximum: a random split
37     would leave the samples with differing shares of expensive houses.
38     """
39     strata = target_bins(y)
40
41     X_tmp, X_test, y_tmp, y_test, s_tmp, _ = train_test_split(
42         X,
43         y,
44         strata,
45         test_size=TEST_SIZE,
46         random_state=SEED,
47         stratify=strata,
48     )
49     X_train, X_val, y_train, y_val = train_test_split(
50         X_tmp,
51         y_tmp,
52         test_size=VAL_SIZE,
53         random_state=SEED,
54         stratify=s_tmp,
55     )
56     return X_train, X_val, X_test, y_train, y_val, y_test

```

Лістинг 1: Поділ вибірки та масштабування (src/data.py)

4. Архітектури нейронних мереж

Відповідно до завдання реалізовано дві архітектури, кожену — у двох варіантах функції активації (ReLU та GELU), що дає чотири конфігурації.

Табл. 2: Реалізовані конфігурації моделей

Модель	Склад	Параметрів
baseline	Dense(64) → Dense(32) → Dense(1), без регуляризації	2 689
regularized	[Dense → BatchNorm → Activation → Dropout(0,2)] ×2 → Dense(1), L2 $\lambda = 10^{-4}$	2 977

```

20 def build_baseline(input_dim: int, activation: str) -> keras.Model:
21     """Baseline feedforward network: two hidden layers, no regularization."""
22     return keras.Sequential(
23         [
24             keras.Input(shape=(input_dim,)),
25             layers.Dense(HIDDEN_UNITS[0], activation=activation),
26             layers.Dense(HIDDEN_UNITS[1], activation=activation),
27             layers.Dense(1),
28         ],
29         name=f"baseline_{activation}",
30     )
31
32
33 def build_regularized(input_dim: int, activation: str) -> keras.Model:
34     """Extended network: Dropout + Batch Normalization + L2 regularization.
35
36     BatchNormalization goes before the activation, which keeps the Dense layer
37     linear and moves the activation out into a layer of its own.
38     """
39     reg = regularizers.l2(L2_LAMBDA)
40     model = keras.Sequential(name=f"regularized_{activation}")
41     model.add(keras.Input(shape=(input_dim,)))
42
43     for units in HIDDEN_UNITS:
44         model.add(layers.Dense(units, kernel_regularizer=reg, use_bias=False))
45         model.add(layers.BatchNormalization())
46         model.add(layers.Activation(activation))
47         model.add(layers.Dropout(DROPOUT_RATE))
48
49     model.add(layers.Dense(1, kernel_regularizer=reg))
50     return model

```

Лістинг 2: Архітектури моделей у TensorFlow (src/tf_models.py)

5. Навчання моделей

Усі моделі навчалися з однаковими гіперпараметрами: оптимізатор **Adam**, швидкість навчання 10^{-3} , розмір батча 256, **20 епох**, функція втрат MSE. Фіксовані зерна генераторів випадкових чисел забезпечують відтворюваність.

```

51 for name, model in models.items():
52     log.info("TensorFlow: %s", name)
53     set_seeds()
54
55     start = time.perf_counter()
56     hist = model.fit(
57         ds["X_train"],
58         ds["y_train"],
59         validation_data=(ds["X_val"], ds["y_val"]),
60         epochs=EPOCHS,
61         batch_size=BATCH_SIZE,
62         verbose=0,
63     )
64     elapsed = time.perf_counter() - start
65
66     pred_val = model.predict(ds["X_val"], verbose=0)
67     pred_test = model.predict(ds["X_test"], verbose=0)
68
69     results[name] = {
70         "val": regression_metrics(ds["y_val"], pred_val),
71         "test": regression_metrics(ds["y_test"], pred_test),
72         "train_time_s": round(elapsed, 2),
73         "n_params": int(model.count_params()),
74         "final_train_loss": float(hist.history["loss"][-1]),
75         "final_val_loss": float(hist.history["val_loss"][-1]),
76     }
77     histories[name] = {
78         "loss": [float(v) for v in hist.history["loss"]],
79         "val_loss": [float(v) for v in hist.history["val_loss"]],
80     }
81
82     r = results[name]
83     log.info(
84         "RMSE(test)=%.4f R2(test)=%.4f time=%.1f s params=%d",
85         r["test"]["rmse"],
86         r["test"]["r2"],
87         elapsed,
88         r["n_params"],
89     )

```

Лістинг 3: Цикл навчання та оцінювання (src/run_lab1.py)

6. Результати

6.1. Метрики якості

Оскільки задача регресійна, використано метрики RMSE, MAE, коефіцієнт детермінації R^2 та MAPE. RMSE і MAE виражені в тих самих одиницях, що й цільова змінна (сотні тисяч доларів), тому допускають пряму інтерпретацію.

Табл. 3: Метрики моделей на тестовій вибірці (TensorFlow)

Модель	RMSE	MAE	R^2	MAPE, %	Час, с
baseline-relu	0,6082	0,4257	0,7191	23,7	7,60
baseline-gelu	0,6181	0,4301	0,7098	24,2	2,29
regularized-relu	0,7480	0,4928	0,5751	23,3	6,84
regularized-gelu	0,7770	0,5126	0,5415	23,6	8,04

Найкращий результат показала базова мережа з активацією ReLU: RMSE = 0,608, що відповідає середній похибці близько 60 тис. доларів, та $R^2 = 0,719$ — модель пояснює майже 72 % дисперсії вартості житла.

6.2. Навчальні криві

Навчальні криві для всіх чотирьох конфігурацій наведено на рисунку 5.

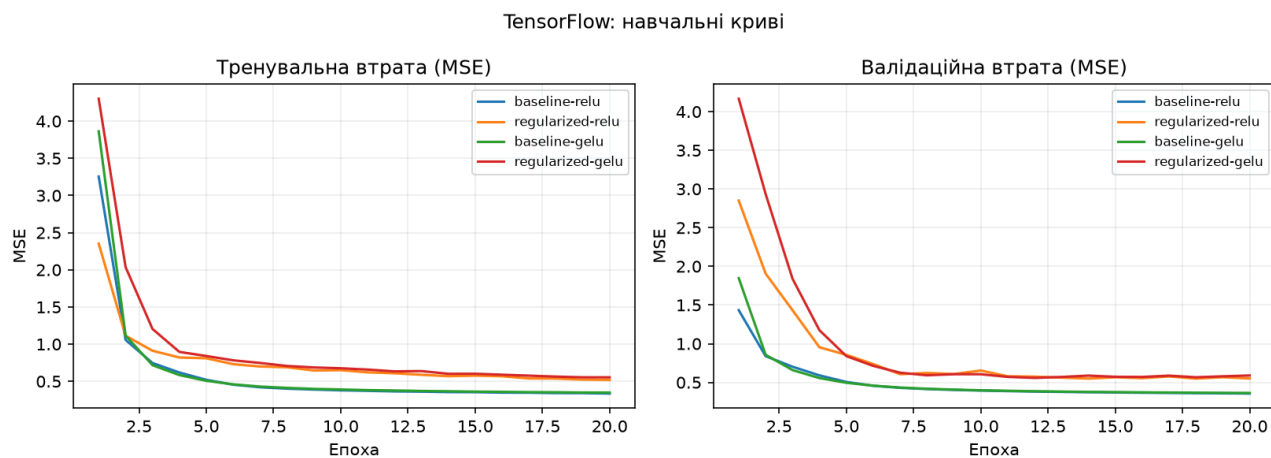


Рис. 5: TensorFlow: криві тренувальної та валідаційної втрати

6.3. Діагностика перенавчання

Різницю між валідаційною та тренувальною втратою зображено на рисунку 6. На другій епосі регуляризовані моделі дають великий сплеск розриву (0,79 та 0,90), однак він швидко спадає: це артефакт шару BatchNorm, ковзні статистики якого на початку навчання ще погано оцінені, через що валідаційна втрата тимчасово завищена.

Починаючи приблизно з сьомої епохи розрив для всіх чотирьох моделей стабілізується і наприкінці навчання не перевищує **0,035**, причому не зростає з часом. Отже, **перенавчання відсутнє**.

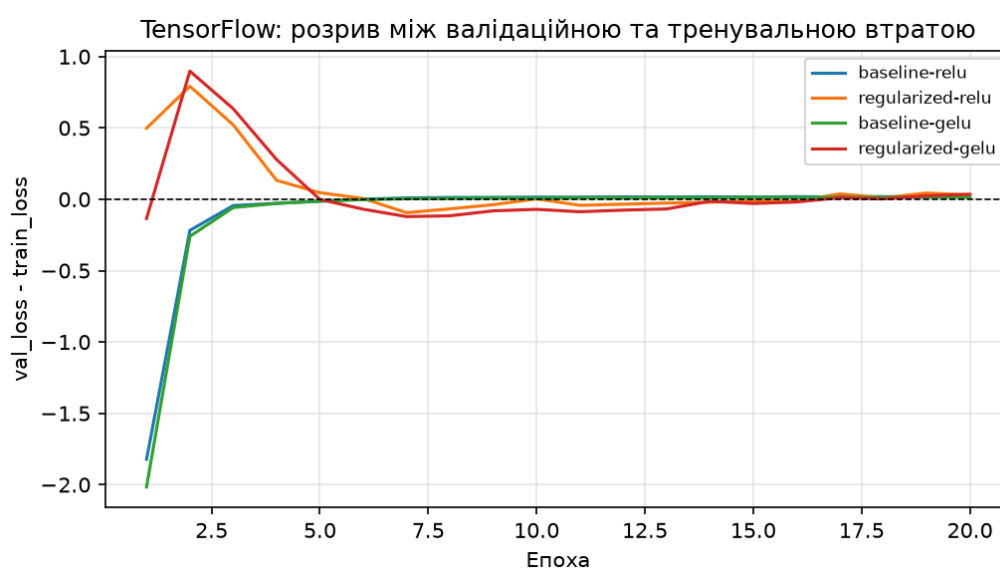


Рис. 6: TensorFlow: розрив між валідаційною та тренувальною втратою

Натомість спостерігається протилежний ефект — **недонавчання регуляризованих моделей**. Їхня втрата висока і на тренувальній, і на валідаційній вибірці (0,520 проти 0,336 у базової моделі з ReLU), що є класичною ознакою надмірної регуляризації, а не перенавчання.

6.4. Якість прогнозу найкращої моделі

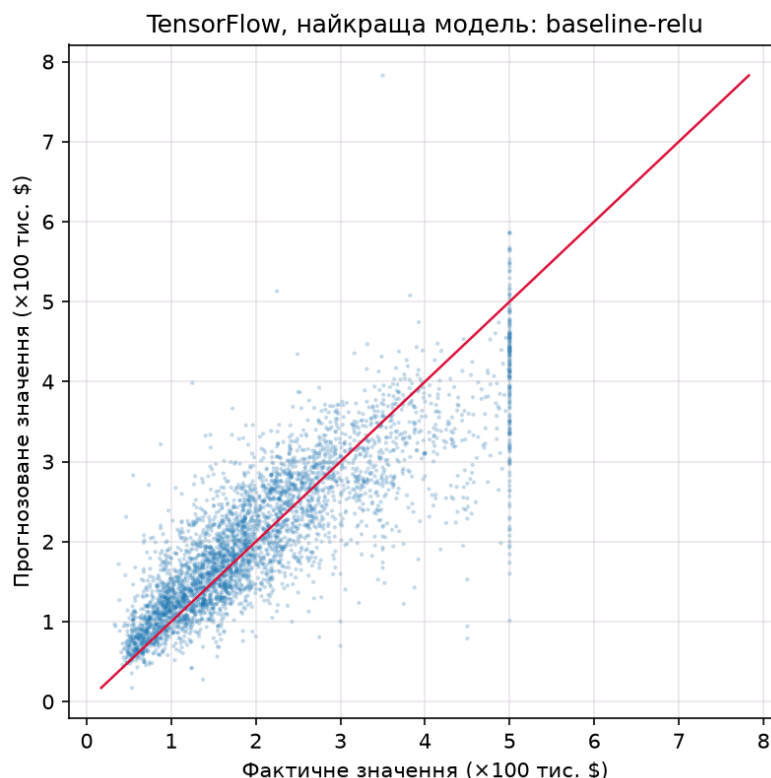


Рис. 7: TensorFlow: прогнозовані значення проти фактичних (baseline-relu)

На рисунку 7 добре помітна вертикальна «смуга» точок при фактичному значенні 5,0 — це саме ті 992 записи з обмеженою зверху вартістю, для яких модель прогнозує широкий діапазон значень. Вони дають істотний внесок у підсумкову похибку, і усунути його засобами моделювання неможливо.

7. Висновки

У ході лабораторної роботи засобами TensorFlow побудовано та проаналізовано чотири конфігурації нейромережових моделей для задачі регресії на наборі даних California Housing.

1. Розвідувальний аналіз показав, що визначальною ознакою є медіанний дохід ($r = 0,689$), а розподіл цільової змінної асиметричний (0,978) та штучно обмежений зверху для 4,81 % записів. Останнє формує нижню межу досяжної похибки.
2. Застосовано стандартизацію ознак і стратифікований за квантилями поділ вибірки 60/20/20. Аналіз PCA показав, що 90 % дисперсії потребує 5 з 8

компонент, тому зменшення розмірності визнано недоцільним.

3. Найкращою виявилася **базова** мережа з активацією ReLU: $RMSE = 0,608$, $R^2 = 0,719$.
4. Регуляризація **погіршила** результат: RMSE зросла з 0,608 до 0,748 (+23 %). Причина — недонавчання: комбінація Dropout 0,2, BatchNorm та L2 надто агресивна для компактної мережі з 2,7 тис. параметрів на 12 тис. тренувальних прикладів. Регуляризація виправдана там, де наявне перенавчання; у цьому експерименті його не було, тому вона лише обмежила здатність моделі апроксимувати залежність.
5. Порівняння функцій активації показало незначну перевагу ReLU над GELU для базової архітектури (RMSE 0,608 проти 0,618). Для цієї задачі вибір активації впливає на результат значно слабше, ніж вибір архітектури.